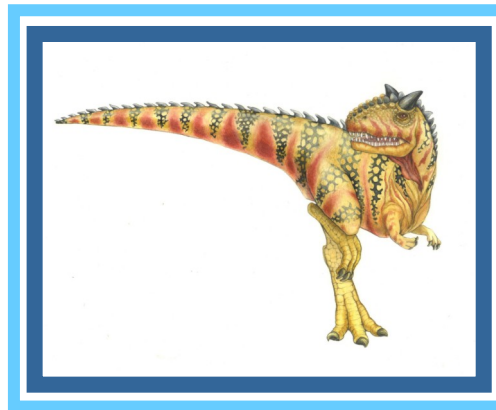


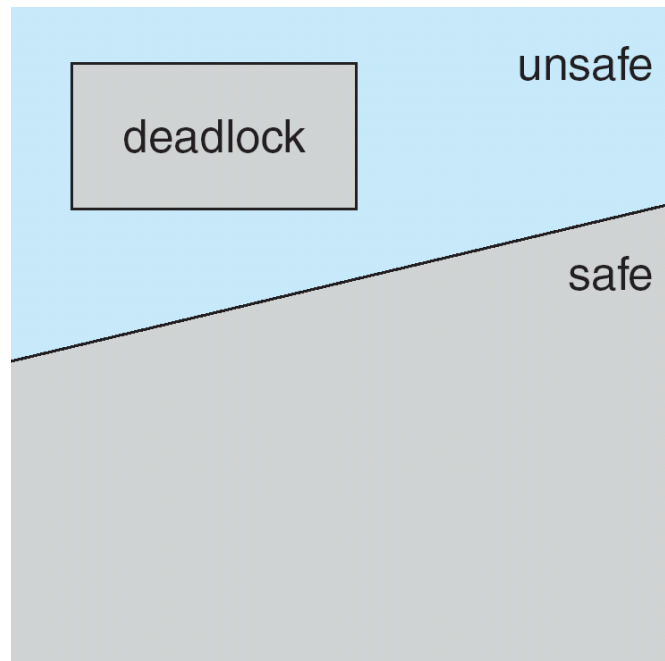
Chapter 6: Deadlocks





Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.





Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in **safe state** if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the systems such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$
- That is:
 - If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
 - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
 - When P_i terminates, P_{i+1} can obtain its needed resources, and so on





Banker's Algorithm

- Multiple instances
- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state.
 - If it will, the resources are allocated, but they must be returned in a finite amount of time;
 - otherwise, the process must wait until some other process releases enough resources.





Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i, j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$





Notations

- Let X and Y be vectors of length n . We say that $X \leq Y$ if and only if $X[i] \leq Y[i]$ for all $i = 1, 2, \dots, n$. For example, if $X = (1, 7, 3, 2)$ and $Y = (0, 3, 2, 1)$, then $Y \leq X$. In addition, $Y < X$ if $Y \leq X$ and $Y \neq X$.
- We can treat each row in the matrices ***Allocation*** and ***Need*** as vectors and refer to them as ***Allocation_i*** and ***Need_i***. The vector ***Allocation_i*** specifies the resources currently allocated to process P_i ; the vector ***Need_i*** specifies the additional resources that process P_i may still request to complete its task.





Safety Algorithm

1. Let **Available** and **Finish** be vectors of length m and n , respectively. Initialize:

Available = Available

Finish [i] = **false** for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

(a) **Finish** [i] = **false**

(b) **Need** _{i} $\leq$ **Available**

If no such i exists, go to step 4

System is in **safe state** if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the systems such that for each P_i , the resources that P_i can still request can be satisfied by **currently available resources + resources held by all the P_j , with $j < i$**

3. **Finish** [i] = **true**

Available = Available + Allocation _{i}
go to step 2

4. If **Finish** [i] == **true** for all i , then the system is in a safe state





Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

- Is the system in a safe state?*





Example (Cont.)

- The content of the matrix **Need** is defined to be **Max – Allocation**

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria





Example (Cont.)

- Why the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria?
- 3 resource types: A (10 instances), B (5 instances), and C (7 instances)

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 4 3	3 3 2
P_1	2 0 0	1 2 2	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Step 1: $avail = [3, 3, 2]$
 $need1 = [1, 2, 2] < avail$
 $finish1 = true$
 $avail = avail + allocation1$
 $= [3, 3, 2] + [2, 0, 0]$
 $= [5, 3, 2]$

Step 2: $need3 = [0, 1, 1] < avail$
 $finish3 = true$
 $avail = avail + allocation3$
 $= [7, 4, 3]$

Step 3: $need4 = [4, 3, 1] < avail$
 $finish4 = true$
 $avail = avail + allocation4$
 $= [7, 4, 5]$

Step 4: $need2 = [1, 2, 2] < avail$
 $finish2 = true$
 $avail = avail + allocation2$
 $= [10, 4, 7]$

Step 5: $need0 = [7, 4, 3] < avail$
 $finish0 = true$
 $avail = avail + allocation0$
 $= [10, 5, 7]$





Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i . If **$Request_i[j] = k$** then process P_i wants k instances of resource type R_j

1. If **$Request_i \leq Need_i$** , go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$Available = Available - Request_i;$

$Allocation_i = Allocation_i + Request_i;$

$Need_i = Need_i - Request_i;$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored





Example: P_1 Request (1,0,2)

- Can we grant P_1 Request (1,0,2)?
- Check that Request $\leq$ Available (that is, (1,0,2) $\leq$ (3,3,2) $\Rightarrow$ true

$$\text{Available} = \text{Available} - \text{Request}_i;$$

$$\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i;$$

$$\text{Need}_i = \text{Need}_i - \text{Request}_i;$$

- So you can get updates as below:

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement. *Why?*





Example: P_1 Request (1,0,2)

- Can we grant P_1 Request (1,0,2)?
- Check that Request $\leq$ Available (that is, (1,0,2) $\leq$ (3,3,2) $\Rightarrow$ true

$$\text{Available} = \text{Available} - \text{Request}_i;$$

$$\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i;$$

$$\text{Need}_i = \text{Need}_i - \text{Request}_i;$$

- So you can get updates as below:

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 3 0	7 2 3	2 1 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- P_0 (0,2,0)





More Exercises

- After P_1 is granted, can request for (3,3,0) by P_4 be granted immediately?
- After P_1 is granted, can request for (0,2,0) by P_0 be granted immediately?



End of Chapter 6

